

An evolutionary programming approach to mixed-variable optimization problems

Y.J. Cao ^{a,*}, L. Jiang ^b, Q.H. Wu ^b

^a *Department of Electrical Engineering, Huazhong University of Science and Technology, Wuhan, Hubei 430074, People's Republic of China*

^b *Department of Electrical Engineering and Electronics, The University of Liverpool, Liverpool L69 3GJ, UK*

Received 29 October 1998; received in revised form 31 March 2000; accepted 23 May 2000

Abstract

Many engineering optimization problems frequently encounter discrete variables as well as continuous variables and the presence of nonlinear discrete variables considerably adds to the solution complexity. Very few of the existing methods can find a globally optimal solution when the objective functions are non-convex and non-differentiable. In this paper, we present a mixed-variable evolutionary programming (MVEP) technique for solving these nonlinear optimization problems which contain integer, discrete, zero-one and continuous variables. The MVEP provides an improvement in global search reliability in a mixed-variable space and converges steadily to a good solution. An approach to handle various kinds of variables and constraints is discussed. Some examples of mixed-variable optimization problems in the literature are tested, which demonstrate that the proposed approach is superior to current methods for finding the best solution, in terms of both solution quality and algorithm robustness. © 2000 Elsevier Science Inc. All rights reserved.

Keywords: Evolutionary programming; Mixed variables; Global optimization; Engineering design optimization

1. Introduction

Many engineering optimization problems frequently encounter discrete variables as well as continuous variables. Discrete variables are used in many ways such as the representation of the set of standard-sized components, the decision on the number of identical parts or the choice between different design options. For example, the number of the teeth of a gear must be chosen as integers; whether or not to select a unit is a zero-one variable; the size of standard diametric pitch of a gear and the capacity of a power unit are discrete variables. The engineering optimal design which contains integer, discrete, zero-one and continuous variables is often referred to as a mixed-variable nonlinear optimization problem. Much research work has been reported on this class of problem. Sandgren and Hajela [1,2] have proposed nonlinear branch and bound algorithms, which are modified from the most widely used method in integer programming. These methods first obtain a solution by dropping the discrete conditions. Cha and Mayne [3] proposed

* Corresponding author. Tel.: +86-27-875 43 327; fax: +86-27-875 40 937.
E-mail address: yjiacao@yahoo.com.cn (Y.J. Cao).

an algorithm which combined a sequential quadratic programming (SQP) method or a local search strategy. The algorithm utilizes recursive quadratic programming but incorporates a Hessian matrix estimation. The algorithm was reported to be efficient based on the test problems, in terms of a low number of objective function and constraint evaluations. One deficiency is that when the objective function is non-convex, the direction for the local search guided by the downhill gradient vector may be wrong and the global optimum will not be reached. Loh and Papalambros [4] introduced a sequential linearization approach for solving mixed-discrete nonlinear optimization problems and test results were provided in [5]. It was reported that this algorithm led to lower number of objective and constraint function evaluations than other major mixed nonlinear algorithms did. For non-convex problems, however, this algorithm might not find the global optimum since the linearization cut off part of the nonlinear feasible region. Fu [6] developed an interior penalty approach, which converts a constrained optimization problem into a sequence of unconstrained problems. The objective function for the unconstrained problem at each step of the sequential optimization includes terms that introduce a penalty depending on the degree of constraint violation. Test results on five problems were provided to demonstrate the solution quality. However, like other penalization-based method, difficulties exist in selecting penalty function parameters and finding the global optimum in some cases.

Among the above methods, no method appears to be substantially better than the others when applicability, efficiency, robustness and solution quality are all considered and very few of them can find a globally optimal solution when the objective functions are non-convex and non-differentiable. For these reasons, a reliable global approach to mixed-variable optimization would be of considerable value to both the optimization and the engineering design community.

Evolutionary programming (EP) is a recently developed stochastic search method [9,10]: it does not require derivatives of the objective function nor assumptions of continuity and convexity of the response surface, and has been successfully applied to complex combinatorial optimization and large-scale systems optimization problems [11–15]. This paper is devoted to developing a mixed-variable evolutionary programming (MVEP) technique for engineering optimization problems, which contain integer, discrete, zero-one and continuous variables. Implementation details of the approach to deal with various types of variables and constraints are discussed. Computational results of the examples on the mixed-variable optimization problems presented show that the MVEP is capable of finding the best solution of mixed-variable optimization problems.

2. Mathematical formulation of the mixed-variable optimization problems

In general, the main objective of the mixed-variable design optimization is to determine the various design variables to minimize an objective function, subject to some design constraints. Let n_c, n_z, n_i and n_d be the number of continuous variables, zero-one variables, integer variables and discrete variables, respectively. The total number of variables is $n = n_c + n_z + n_i + n_d$, then the mixed-variable optimization problem may be expressed in mathematical form as follows:

$$\begin{aligned} \min \quad & f(X) \\ \text{s.t.} \quad & h_i(X) = 0, \quad i = 1, 2, \dots, m, \\ & g_i(X) \leq 0, \quad i = m + 1, \dots, p, \end{aligned} \tag{1}$$

$$X = \begin{pmatrix} X^c \\ X^z \\ X^i \\ X^d \end{pmatrix} = \left[x_1^c, \dots, x_{n_c}^c, x_1^z, \dots, x_{n_z}^z, x_1^i, \dots, x_{n_i}^i, x_1^d, \dots, x_{n_d}^d \right]^T,$$

$$x_i^{cl} \leq x_i^c \leq x_i^{cu}, \quad i = 1, 2, \dots, n_c.$$

$$x_i^{zl} \leq x_i^z \leq x_i^{zu}, \quad i = 1, 2, \dots, n_z,$$

$$x_i^{il} \leq x_i^i \leq x_i^{iu}, \quad i = 1, 2, \dots, n_i,$$

$$x_i^{dl} \leq x_i^d \leq x_i^{du}, \quad i = 1, 2, \dots, n_d,$$

where $X^c \in R^c$, $X^z \in R^z$, $X^i \in R^i$ and $X^d \in R^d$ denote feasible subset of continuous, zero-one, integer and discrete variables, respectively. x_i^{cl} , x_i^{zl} , x_i^{il} and x_i^{dl} are the lower bounds of i th continuous variable, zero-one variable, integer variable and discrete variable, respectively. x_i^{cu} , x_i^{zu} , x_i^{iu} and x_i^{du} are the upper bounds of i th continuous variable, zero-one variable, integer variable and discrete variable, respectively. Here $x_i^{zl} = 0$ and $x_i^{zu} = 1$.

The above optimization problem is basically the same as the general nonlinear programming except that the design variables may take on any form of continuous, zero-one, integer and discrete variable. Zero-one variables are usually required in the formulation of design problem with alternative option selections. Integer variables are used to formulate design problems with a number of objects, such as the number of gear teeth. Discrete variables are needed to represent standard parameters, for example, the diameter of a standard sized bolt and the capacity of a standard power unit. It has been shown that the presence of nonlinear discrete variables considerably adds to the solution complexity [2,3].

3. Evolutionary programming

EP intrinsically provides a robust random search and optimization mechanism. Random optimization has traditionally been based on single-agent stochastic search (SASS) strategies. Both Rao and Karnop [16,17] generate a random walk sequence to an extremum by perturbing the search point with a uniform random variable. Rao also exploits the directionality of the randomly generated vectors that continue to yield lower valued objective functions. In an algorithmic formulation similar to that of Rao, Matyas utilizes Gaussian perturbations about the search point along with a bias term to direct the search [18]. Solis and Wets [19] have enhanced this approach by evaluating the objective function at $x - \delta x$ if the evaluation at $x + \delta x$ does not improve the current value of the objective function and by incorporating a variable perturbation variance. The bias and additional function evaluation serve as stochastic equivalents to incorporating momentum and gradient information.

Due to the availability of the computers to evaluate the objective function quickly at many points, EP maintains a population of search points instead of keeping the single best point, the way which was used in the 1950s and 1960s. So EP is a systematic population-based random search technique that can be used for finding global extrema on the rugged response surface. It starts with a randomly generated population and evolves towards a better solution by applying the mutation, competition and selection operators, modeled on the evolutionary process occurring in nature. The mutation operator is done by applying Gaussian distributed perturbations to

each element in the solution vector. Furthermore, EP employs a competition process, in which an individual competes with some other individuals in the population and the mutated old generation. Winners of the same number as the individuals in the old generation form the next generation. This allows less fit individuals (search points) to be retained in the population in a probabilistic fashion. This is originally similar to the *creeping random method* (CRM), proposed by [20], in which k points were generated via Gaussian perturbations about a search point. The best point was kept and the process repeated. Brooks observed that ‘there are some rather intriguing analogies that can be made between the creeping random and evolution’. The main

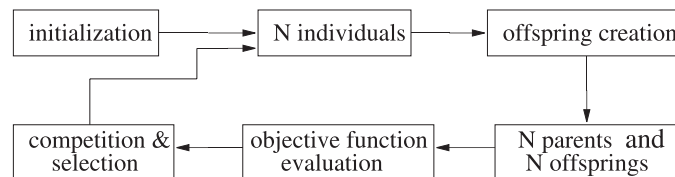


Fig. 1. Schematic diagram of EP algorithm.

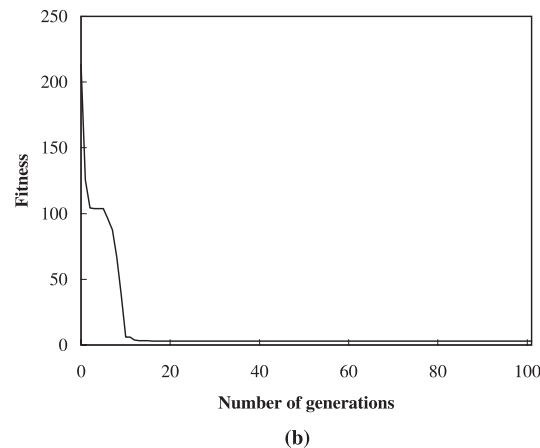
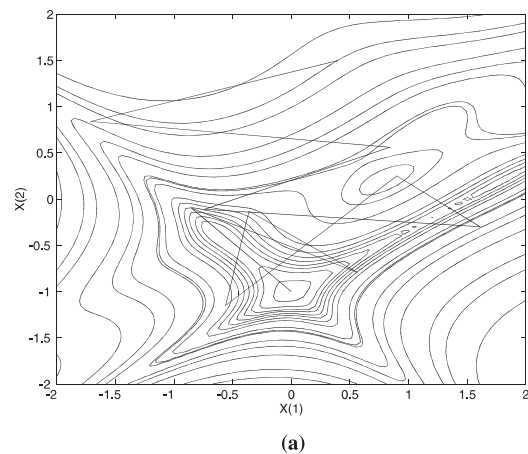


Fig. 2. EP algorithm on GP function.

difference between EP and CRM is that in CRM, only the single best point was kept, whilst in EP, a population of search points is maintained.

The standard EP procedure for real-valued optimization problems, outlined by Fogel [10], is shown in Fig. 1. Here, we assume that the minimal objective value is positive, otherwise a simple normalization process described in the next section should be employed. For the function minimization problem, the EP procedure is conducted as follows:

1. Select an initial population of N trial solutions, x_i , $i = 1, \dots, N$, by sampling from a uniform distribution across the preselected range of each variable.
2. Score all parent solutions x_i , $i = 1, \dots, N$, with respect to the chosen function $f(x)$.
3. From each parent x_i , $i = 1, \dots, N$, create an offspring, denoted as x_{i+N} , by adding a Gaussian random variable with zero mean and positive variance set proportional to $f(x_i)$ to each component of the parent.
4. Score all offspring x_i , $i = N + 1, \dots, 2N$, with respect to the chosen function $f(x)$.
5. For each x_i , $i = 1, \dots, 2N$, select s competitors at random from the population. Conduct pairwise comparisons between x_i and each of the competitors. If the $f(x)$ of x_i is less than or equal to that of its opponent, assign it a win.
6. Select the N solutions that have the largest number of wins to be parents for the next generation.
7. If the available computing time has expired then halt, else return to step 3.

A trajectory of the best population member at each generation during a search on the Goldstein and Price (GP) function [21] is superimposed on the GP contours shown in Fig. 2(a). Fig. 2(b) shows the best objective value in the population at each generation of the EP optimization process. The search is stochastic, so it is expected that this trajectory will vary in every trial.

4. Mixed-variable evolutionary programming

The MVEP algorithm for global optimization contains four parts: initialization, mutation, competition and selection. For the design optimization problem, various types of variables are involved, and some modifications and treatments to the standard EP are needed. The primary difference between the MVEP and the standard EP is in the form of the mutation operator. The mutation operator should be constructed carefully for each of the different types of variable. The MVEP algorithm implemented to solve the design optimization problem is stated as follows.

4.1. Initialization

The initial parent trial vector x_i , $i = 1, 2, \dots, N$, is determined by setting its components as

$$\begin{aligned} x_j^c &\sim \text{UC}(x_j^{\text{cl}}, x_j^{\text{cu}}), & j &= 1, 2, \dots, n_c; & x_j^z &\sim \text{UI}(x_j^{\text{zl}}, x_j^{\text{zu}}), & j &= 1, 2, \dots, n_z; \\ x_j^i &\sim \text{UI}(x_j^{\text{il}}, x_j^{\text{iu}}), & j &= 1, 2, \dots, n_i; & x_j^d &\sim \text{UD}(x_j^{\text{dl}}, x_j^{\text{du}}), & j &= 1, 2, \dots, n_d, \end{aligned}$$

where $\text{UC}(a, b)$, $\text{UZ}(a, b)$, $\text{UI}(a, b)$ and $\text{UD}(a, b)$ denote uniform random continuous variable, zero-one variable, integer variable and discrete variable ranging over $[a, b]$, respectively. N individuals are randomly generated. Each individual is checked by a validation procedure (see *treatment of constraints*) which ensures that only a feasible individual can be placed in the initial population.

4.2. Normalization process for objective values

EP is originally used to solve optimization problems with positive objective function values. For those minimization problems with negative objective function values, the following simple normalization process is employed:

$$f'(x_i) = b + f(x_i), \quad (2)$$

where $f'(x_i)$ is the normalized objective value for individual x_i , and used for mutation and selection operators, $f(x_i)$ is the objective function value for the individual x_i , and b is a constant chosen such that $f'(x_i) > 0$. So b is usually chosen relatively large and by trial-and-error method.

4.3. Mutation

A parent vector x_i is mutated to create an offspring vector x_{i+N} , based on the following rule:

1. *For continuous variables:* A mutation is carried out by adding to each component of $x_{i,j}^c$ a Gaussian variable with zero mean and a standard deviation proportional to the normalized objective value of the parent trial solution, that is

$$\begin{aligned} x_{i+N,j}^c &= x_{i,j}^c + \mathcal{N}(0, \sigma_j^2), \quad j = 1, 2, \dots, n_c, \\ \sigma_j^2 &= \beta_j \frac{f'_i}{f'_{\min}}, \end{aligned} \quad (3)$$

where $x_{i,j}^c$ denotes the j th element of the continuous variable in i th individual; $\mathcal{N}(\mu, \sigma^2)$ represents a Gaussian random variable with a mean μ and variance σ^2 ; $f'_{\min}$ is minimal normalized objective value of the individuals in the population; β_j is a constant of proportionality to scale $f'_i/f'_{\min}$.

2. *For zero-one variables:* A mutation is conducted by uniformly changing a variable from zero to one and vice versa, that is

$$x_{i+N,j}^z = \begin{cases} 1 & \text{if } x_{i,j}^z = 0 \text{ and } r_1 \leq 0.5, \\ 0 & \text{if } x_{i,j}^z = 0 \text{ and } r_1 > 0.5, \\ 0 & \text{if } x_{i,j}^z = 1 \text{ and } r_1 \leq 0.5, \\ 1 & \text{if } x_{i,j}^z = 1 \text{ and } r_1 > 0.5, \end{cases} \quad j = 1, 2, \dots, n_z, \quad r_1 \sim U(0, 1).$$

3. *For integer variables:* A mutation is achieved by changing a variable from one integer value to another one with a random step size IST_j , that is

$$x_{i+N,j}^i = x_i^i + \text{INT}[(2r_2 - 1) \cdot IST_j], \quad j = 1, 2, \dots, n_i, \quad r_2 \sim U(0, 1).$$

4. *For discrete variables:* A mutation is done by changing a variable from a prespecified discrete value to another one, that is

$$x_{i+N,j}^d = d_{(l + \text{INT}[(2r_3 - 1) \cdot DST_j])} \quad \text{if } x_i^d = d_l, \quad j = 1, 2, \dots, n_d, \quad r_3 \sim U(0, 1), \quad (4)$$

where IST_j is a preset step size (an integer) of integer variable for a mutation, DST_j is a step size (an integer) of discrete variable for a mutation, d_l is the l th element of the discrete variable subset X^d and $\text{INT}[x]$ denotes the greatest integer less than the real value x .

4.4. Competition and selection

Each x_{i+N} , $i = 1, 2, \dots, N$, is again assigned a score f'_{i+N} . Based on the mutated population with the size of $2N$, a competition is conducted to reproduce offsprings. For each x_i , $i = 1, 2, \dots, 2N$, a value W_i is assigned to weight the individual according to the following equation:

$$W_i = \sum_{t=1}^s w_{t,i} \quad (5)$$

and

$$w_{t,i} = \begin{cases} 1 & \text{if } r_4 < f'_{v_t} / (f'_{v_t} + f'_i), \\ 0 & \text{otherwise.} \end{cases} \quad (6)$$

where s is the number of competitors, $v_t = \text{INT}[2Nr_5 + 1]$, and $r_4, r_5 \sim U(0, 1)$. s is a parameter chosen to control the competition process which involves generating s random numbers v_t , and s times comparison between the randomly chosen individual v_t and the individual i . If s is chosen to be equal to the population size, the whole competition process is similar to ordering the individuals based on their objective value. s is usually chosen to be less than the population size and a good value of s is around half of population size. The individuals x_i , $i = 1, 2, \dots, 2N$, are ranked in descending order of their corresponding value W_i . The first N individuals are transcribed along with their corresponding normalized objective value f'_i to be the basis of the next generation.

4.5. Treatment of constraints

Most engineering design optimization problems are constrained. Two techniques are often used to handle constraints. One is a variable restriction method which restricts the solution space to the solutions which conform to the constraints. The other is a penalty function method which allows solutions which violate the constraints at the expense of a suitably defined penalty

```

begin
  Initialize;
  Statistics;
  number_of_gen := 0;
  number_of_convergence := 0;
  repeat
    number_of_generation := number_of_generation + 1;
  repeat
    mutation based on eqn. (3) ~ (4),
  until the new solution satisfies the constraints;
  Competition and Selection;
  Statistics;
  if maxfitness - minfitness < ε then
    number_of_convergence := number_of_convergence + 1;
    oldpopulation := newpopulation;
    until (number_of_generation = maxnumber_of_generation ) or
      (number_of_convergence = maxnumber_of_convergence);
  end.

```

Fig. 3. Procedure of the mixed-variable evolutionary programming.

function. However, determining appropriate penalty coefficients is not an easy task, it must be estimated based on the relative scaling of the distance metrics of multiple constraints, the difficulty of satisfying a constraint, and the seriousness of a constraint violation, or be determined experimentally. Although the adaptive penalty strategies proved to be effective in some cases, they are still quite problem-dependent [22]. It is also noticed that if the values of the objective function are very large, then small differences between objective values are not easily identified, and this is undesirable, a leading cause of premature convergence [10] in EP.

In this study, the variable restriction method is used, in which a mutation is first randomly done, and the solution is then checked against the constraints. If the constraints are satisfied, the mutation is usable, otherwise, the mutation is discarded and a new mutation is generated and checked. It should be noted that some of the mutations generated are unusable, and therefore more computational time may be needed.

The implementation of MVEP for the mixed-variable design optimization problems is described briefly in pseudo-code as in Fig. 3.

5. Computation examples

The MVEP algorithm was coded and tested on a number of mixed-variable nonlinear constrained optimization problems. These problems were chosen from those given in Refs. [1,7,8]. A brief description of the test problems, which includes the name of problem, the number of variables, the number of discrete variables, the number of constraints and the reference number, is summarized the column headed *description* in Table 1. For example, (Gupta5, 4 2 5 7) denotes that the problem is called Gupta5 in the literature, has 4 variables, 2 discrete variables (including integer variables), 5 constraints and comes from [7]. Most of problems (1–11) were randomly generated by the previous researchers. Some of them are highly nonlinear and non-convex. Therefore, solution spaces of these problems is likely to be very complex. The others are real-world engineering design problems which have appeared in the literature.

This section is divided into two parts. In the first one, the best objective values obtained by MVEP with a typical parameter setting are presented and compared to those obtained by

Table 1
Comparison of the best objective values by different methods

Problem	Description	DRQP	MINLP	MIBBGRG	MVEP
1	(Gupta5, 4 2 5 7)	5.476	5.47	5.471	5.47
2	(Gupta8, 3 2 3 7)	^a	23.45	23.45	20.46 ^b
3	(Gupta17, 7 7 7 7)	^a	−1067.0	−1100.4	−1100.4
4	(Gupta18, 6 6 6 7)	^a	−767.8	−778.4	−778.8 ^b
5	(Gupta19, 8 8 8 7)	^a	−1082.8	−1098.4	−1098.4
6	(Gupta21, 3 2 2 7)	^a	−56847.83	−56847.83	−56847.83
7	(Cha1, 2 2 2 8)	13.387	13.387	13.387	13.387
8	(Cha3, 5 5 6 8)	−1436.7	−1436.7	−1436.7	−1436.7
9	(Cha8, 5 5 10 8)	−32.3	−32.3	−32.3	−32.3
10	(Cha12, 5 4 7 8)	47005.3	42866.7	45543.9	42259.7 ^b
11	(Cha15, 6 6 4 8)	4.098	4.279	4.279	3.218 ^b
12	(Bolt selection, 4 4 4 8)	^a	306	306	306
13	(Gear train, 4 4 0 1)	^a	2.33×10^{-7}	^a	2.7×10^{-12} ^b
14	(Sandgren's vessel, 4 2 4 1)	^a	7197.7	^a	7108.6 ^b
15	(Loh's pressure vessel, 5 5 22 8)	^a	47818.43	47818.43	39977.72 ^b

^a Denotes the result was not reported.

^b Indicates better result was obtained by MVEP.

previous existing methods. In the second part, the sensitivity of MVEP parameters is examined and discussed.

5.1. Comparison with previous optimization methods

In this study, the MVEP was compared with several existing algorithms for mixed-discrete optimization. These candidate algorithms are: discrete recursive quadratic programming (DRQP) algorithm [3], mixed integer nonlinear sequential linearization programming (MINSLP) algorithm [4] and mixed integer branch and bound using the generalized reduced gradient (MIBBGRG) algorithm [7]. For some of the above algorithms, only some of the test problems were solved and reported. For a test problem, the comparison was made only between the algorithms which have reported on that particular test problem.

In the MVEP algorithm, the population size, the number of competitors, the scaling coefficient, the step size for integer variables and discrete variables were chosen as 50, 20, 0.5, 3 and 2 respectively. The optimal objective function values, obtained by different candidate algorithms and MVEP algorithm are summarized in Table 1. From the results shown in the table, it can be seen that the MVEP algorithm performed very well. For all test problems, the MVEP algorithm was able to find the best known solution. For seven test problems (2, 4, 10, 11, 13, 14, 15), MVEP found better solutions than best known reported. The detailed solutions for these problems are provided in Table 2, where X^* is the minimizer. For problems (10, 11, 13, 15), much better solutions were found by MVEP. MVEP found the best solution at (12, 0.6875, 44, 0.3125, 0.7126) with $f = 42259.7$ for problem 10. DRQP, MINSLP and MIBBGRG reported different solutions with different optimum objective values: 47005.3, 42866.7 and 45543.9. Because of the non-convexity of the problem, the local search in DRQP appeared to be unable to jump out from the local minimum. The reason for MINSLP's failing to reach the global optimum was that the linearization cut off the true minimizer. MIBBGRG failed to obtain the global optimum, because it fathomed a node for the non-convex problem.

5.2. Sensitivity of main MVEP parameters

Five parameters characterizing the proposed MVEP have an important influence on its convergence, namely the population size (N), the number of competitors (s), the scaling coefficient of continuous variables (β_j), the step sizes for integer variables (IST_j) and discrete variables (DST_j). To avoid any hazardous interpretation of optimization results, related to the choice of particular initial populations, we performed each test 30 times, starting from different populations with individuals randomly generated in the search domain. Indeed MVEP being partly a stochastic algorithm, results yielded by one single execution cannot be considered as trustworthy. Therefore,

Table 2
Detailed solutions obtained by different methods

Problem	MINLP (X^*)	MIBBGRG (X^*)	MVEP (X^*)
2	(4, 3, 0.631) ^T	(4, 3, 0.631) ^T	(3, 3, 0.513) ^T
4	(2, 7, 3, 2, 7, 5) ^T	(2, 6, 3, 2, 8, 6) ^T	(3, 6, 3, 2, 8, 5) ^T
10	(10, 0.5, 55, 0.375, 0.733) ^T	(12, 0.6875, 44, 0.3125, 0.713) ^T	(12, 0.6875, 44, 0.3125, 0.7126) ^T
11	(9, 0.66, 0, 0, 13) ^T	(9, 0.66, 0, 0, 13) ^T	(288, 0, 0, 0, 28, 13) ^T
13	(19, 16, 42, 50) ^T	—	(19, 16, 49, 43) ^T
14	(1.125, 0.625, 58.290, 43.693) ^T	—	(1.000, 0.625, 51.196, 90.782) ^T
15	(0.3125, 0.1875, 0.5625, 1.125, 0.3125) ^T	(0.3125, 0.1875, 0.5625, 1.125, 0.3125) ^T	(0.3125, 0.1875, 0.5625, 1.0625, 0.3125) ^T

the MVEP tests were systematically performed 30 times, with different initial seeds for the pseudo-random numbers generator, and hence different initial populations. To appraise the MVEP's efficiency, we retained the following criteria summarizing results from 30 minimizations, for each test problem: standard deviation of the final 'errors', defined as objective values gaps between best successful individual found and the known global optimum, average of objective function evaluations numbers, and the number of unusable mutations.

Sensitivity of the population size and the number of competitors. As discussed in the above section and suggested in Refs. [10,14,27], a good choice of the number of competitors is around half of the population size, this is because if the number of competitors is chosen to be too large, say, is equal to the population size, the whole competition process is similar to ordering the individuals based on their objective values, which produces too much selection pressure and easily results in premature convergence. So in this study, the number of competitors is chosen around half of the population size. We tested various values of population size between 20 and 150. In the interest of brevity, only results for problems (10, 14, 15), in this case $\beta_j = 5$, $IST_j = 3$ and $DST_j = 2$, are reported in Table 3. The numbers of unusable mutations is strongly correlated to the population size. But the accuracy of the result and the speed of MVEP convergence are weakly dependent on the population size, provided that it is not chosen too small (less than 30). Roughly speaking, a large population size (more than 100) induces numerous unusable mutations, which is costly in CPU time. In the opposite case, a too small population size (less than 30) implies bad accuracy of the results and speed of convergence. For the problems under test, the best compromise seems to be reached around 50.

Sensitivity of the scaling coefficient of continuous variables and the step sizes for integer variables and discrete variables. We tested various values of the scaling coefficient of continuous variables between 0.0 and 1.0, and the step sizes for integer variables and discrete variables between 1 and 9. Results observed for problems (10, 14, 15), in this case the population size and the number of competitors being 50 and 20, are reported here in Table 4. Usually, large scaling coefficient of continuous variables and step sizes for integer variables and discrete variables can easily induce numerous unusable mutations, on the other hand, too small scaling coefficient of continuous variables and step sizes for integer variables and discrete variables implies more objective function evaluations needed. Their optimal values, suggested for the test problems, are $\beta_j = 0.5$, $IST_j = 3$ and $DST_j = 3$.

Table 3
Sensitivity of the population size (N) and the number of competitors (s)

(N, s)	Problem	Standard deviation of final 'errors'	Average number of objective evaluations	Average number of unusable mutations
(20, 10)	10	0.015	2846	217
	14	0.180	2874	341
	15	0.034	3626	587
(50, 20)	10	0.006	1524	403
	14	0.012	1612	621
	15	0.008	2018	1124
(100, 40)	10	0.006	1412	987
	14	0.010	1484	1384
	15	0.007	1858	2317
(150, 60)	10	0.008	1377	1707
	14	0.010	1424	2443
	15	0.006	1772	5112

Table 4

Sensitivity of the scaling coefficient of continuous variables and the step sizes for integer variables and discrete variables

$(\beta_j, \text{IST}_j, \text{DST}_j)$	Problem	Standard deviation of final 'errors'	Average number of objective evaluations	Average number of unusable mutations
(0.2, 1, 1)	10	0.006	2621	302
	14	0.011	3112	467
	15	0.008	4127	624
(0.4, 4, 3)	10	0.006	1441	412
	14	0.010	1622	578
	15	0.007	1887	1022
(0.6, 6, 4)	10	0.008	1642	442
	14	0.012	1728	582
	15	0.008	2034	1183
(0.8, 8, 8)	10	0.010	1822	1604
	14	0.012	2123	2382
	15	0.08	2310	4225

With the parameters properly chosen, a set of computational experiments were carried out for solving each test problem from different initial populations. Our observations show that MVEP behaved highly robustly for all of the test problems. The algorithm was able to reach the same solution from different initial populations. Some existing algorithms, like MINSLP and MIBBGRG, were reported to be dependent on initial solutions. For MVEP, the setting of algorithm parameters had some effects on the solution quality. It is a good practice to find a set of appropriate parameters. Based on our experience, the population size is normally chosen between 30 and 100. For continuous variable, the scaling coefficient should be kept relatively small (0.3–0.6), while for discrete and integer variables, the mutation step size could be a larger integer (2–6).

6. Conclusions

MVEP is an efficient stochastic optimization technique which is capable of handling different kinds of variables and the constraints involved and locating the optimal solution in the rugged response surface. In this paper, an MVEP algorithm is developed for engineering design optimization problems containing integer, discrete, zero-one and continuous variables. The approach to the diverse variable representations and constraints has been discussed. Some test examples are given to show that the developed MVEP algorithm is superior to the existing methods of this kind for finding the best solution, both in the solution quality and algorithm robustness. Future research should concentrate mainly on developing self-adaptation techniques [23–27] for the mutation parameters and penalty functions to reduce the number of unusable mutations and improve the solution quality.

Acknowledgements

The authors would like to thank Dr. D.W. Shimmin and the referee for their valuable comments. The first author would like to appreciate the financial support from NSF and SRF for ROCS, SEM in the People's Republic China.

References

- [1] E. Sandgren, Nonlinear integer and discrete programming in mechanical design optimization, *J. Mech. Design* 112 (1990) 223–229.
- [2] P. Hajela, C. Shih, Multiobjective optimum design in mixed-integer and discrete design variable problems, *AIAA J.* 28 (1989) 670–675.
- [3] J. Cha, R. Mayne, Optimization with discrete variables via recursive quadratic programming: part II, *Trans. ASME* 111 (1989) 130–136.
- [4] H.T. Loh, P.Y. Papalambros, A sequential linearization approach for solving mixed-discrete nonlinear design optimization problems, *ASME J. Mech. Design* 113 (1991) 325–334.
- [5] H.T. Loh, P.Y. Papalambros, Computational implementations and tests of a sequential linearization algorithm for mixed-discrete nonlinear design optimization problems, *ASME J. Mech. Design* 113 (1991) 335–345.
- [6] J.F. Fu, R.G. Fenton, W.L. Cleghorn, A mixed integer-discrete-continuous programming method and its application to engineering design optimization, *Eng. Optim.* 17 (1991) 263–280.
- [7] O.K. Gupta, Branch and bound experiments in nonlinear integer programming, Ph.D. Dissertation, Purdue University, W. Lafayette, 1980.
- [8] H.T. Loh, A sequential linearization approach for mixed-discrete nonlinear design optimization, Doctoral Dissertation, The University of Michigan, Ann Arbor, 1989.
- [9] D.B. Fogel, An introduction to simulated evolutionary optimization, *IEEE Trans. Neural Network* 5 (1994) 3–14.
- [10] D.B. Fogel, *Evolutionary Computation: Toward a New Philosophy of Machine Intelligence*, IEEE Press, New York, 1995.
- [11] D.B. Fogel, Applying evolutionary programming to selected travelling salesman problems, *Cybern. Syst.: An Int. J.* 24 (1993) 27–36.
- [12] Y.J. Cao, Q.H. Wu, D.W. Shimmin, Study of initial population in evolutionary programming, in: *Proceedings of European Control Conference ECC '97*, Brussels, 1997, Paper no: 368, pp. 1–4.
- [13] Y.J. Cao, Eigenvalue optimisation problems via evolutionary programming, *Electron. Lett.* 33 (1997) 642–643.
- [14] Y.J. Cao, Q.H. Wu, An improved evolutionary programming approach to economic dispatch, *Int. J. Eng. Intell. Sys.* 6 (1998) 187–194.
- [15] Y.J. Cao, Q.H. Wu, Applying evolutionary programming to robust systems analysis and control, in: *Proceedings of IEEE World Congress on Evolutionary Computation*, vol. 3, 1999, pp. 2109–2115.
- [16] S.S. Rao, *Optimization Theory and Applications*, Wiley, New York, 1978.
- [17] D.C. Karnopp, Random search techniques for optimization problems, *Automatica* 1 (1963) 111–121.
- [18] J. Matyas, Random optimization, *Autom. Remote Control* 26 (1965) 244–251.
- [19] F.J. Solis, J.B. Wets, Minimization by random search techniques, *Math. Oper. Res.* 6 (1981) 19–50.
- [20] S.H. Brooks, A discussion of random methods for seeking maxima, *Oper. Res.* 6 (1958) 244–251.
- [21] A. Dekkers, E. Aarts, Global optimization and simulated annealing, *Math. Prog.* 50 (1991) 367–393.
- [22] D.W. Coit, A.E. Smith, D.M. Tate, Adaptive penalty methods for genetic optimization of constrained combinatorial problems, *INFORMS J. Comput.* 8 (1996) 173–182.
- [23] J.J. Grefenstette, Optimization of control parameters for genetic algorithms, *IEEE Trans. Syst. Man Cybern.* 16 (1986) 122–128.
- [24] Q.H. Wu, Y.J. Cao, J.Y. Wen, Optimal reactive power dispatch using an adaptive genetic algorithm, *Int. J. Elect. Power Energy Syst.* 20 (1998) 563–569.
- [25] Y.J. Cao, Q.H. Wu, Optimisation of control parameters in genetic algorithms: a stochastic approach, *Int. J. Syst. Sci.* 30 (1999) 551–559.
- [26] H.P. Schwefel, *Numerical Optimization of Computer Models*, Wiley, Chichester, UK, 1981.
- [27] D.B. Fogel, L.J. Fogel, J.W. Atmar, Meta-evolutionary programming, in: *Proceedings of the 25th Asilomar Conference on Signals, Systems, and Computers*, Maple Press, Pacific Grove, CA, 1991, pp. 540–545.